23Z711 - PROJECT WORK 1

UW-TIG: Uncertainty-Weighted Temporal Information Gain for Closed-Loop Defect Inspection

A Novel Active-Reinspection Planner and Its Simulation Testbed

Akhil Ramalingam	(Roll No: 23Z207)
Anbuchandiran K	(Roll No: 23Z209)
Neelesh Padmanabh	(Roll No: 23Z241)
Saumiyaa Sri V L	(Roll No: 23Z261)
Sowndarya Elangi S	(Roll No: 23Z269)
Therdhana J P	(Roll No: 23Z273)

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch : COMPUTER SCIENCE and ENGINEERING

of Anna University



NOVEMBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

PSG COLLEGE OF TECHNOLOGY

(Autonomous Institution)

COIMBATORE – 641 004

UW-TIG: Uncertainty-Weighted Temporal Information Gain for Closed-Loop Defect Inspection

Bona fide record of work done by

Akhil Ramalingam	(Roll No: 23Z207)
Anbuchandiran K	(Roll No: 23Z209)
Neelesh Padmanabh	(Roll No: 23Z241)
Saumiyaa Sri V L	(Roll No: 23Z261)
Sowndarya Elangi S	(Roll No: 23Z269)
Therdhana J P	(Roll No: 23Z273)

Dissertation submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

Branch : COMPUTER SCIENCE and ENGINEERING

of Anna University

NOVEMBER 2026

.....	.....
Ms. L. Karthika	Dr. N. Gopika Rani
Faculty guide	Co-guide

.....

Dr. G. R. Karpagam

Head of the Department

Certified that the candidates were examined in the viva-voce examination held on .....

.....	.....
(Internal Examiner)	(Panel Members)

CERTIFICATE

Certified that this report titled "**UW-TIG: Uncertainty-Weighted Temporal Information Gain for Closed-Loop Defect Inspection**" for the Project Work I (23Z711) is a bonafide work of **Akhil Ramalingam (23Z207)**, **Anbuchandiran K (23Z209)**, **Neelesh Padmanabh (23Z241)**, **Saumiyaa Sri V L (23Z261)**, **Sowndarya Elangi S (23Z269)**, **Therdhana J P (23Z273)** who have carried out the work under my supervision for the partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation based on which a degree or an award was conferred on an earlier occasion.

Place: Coimbatore

Date: 02.11.2026

Members

Akhil Ramalingam
Anbuchandiran K
Neelesh Padmanabh
Saumiyaa Sri V L
Sowndarya Elangi S
Therdhana J P

Faculty Guide

Ms. L. Karthika
Department of Computer Science and Engineering
PSG College of Technology
Coimbatore - 641004

Co-Guide

Dr. N. Gopika Rani
Department of Computer Science and Engineering
PSG College of Technology
Coimbatore - 641004

COUNTERSIGNED

Dr. G. R. Karpagam
HEAD
Department of Computer Science and Engineering
PSG College of Technology
Coimbatore – 641004

CONTENTS

1. INTRODUCTION 2

1.1. Objective 2

1.2. Overview 3

1.3. Scope 3

2. LITERATURE SURVEY 4

2.1. Literature Survey 4

2.2. Inference 6

2.3. Research Gap 7

2.4. Research Objectives 7

3. SYSTEM REQUIREMENTS 8

3.1. Hardware Requirements 8

3.2. Software Requirements 8

3.3. Datasets 9

4. SYSTEM DESIGN 10

4.1. Overall Architecture 10

4.2. Simulation Environment 10

4.3. Two Simulation Modes 13

4.4. Candidate Viewpoint Graph 13

4.5. Scenarios 14

4.6. Perception 14

4.7. 3-D Localisation 15

4.8. Persistent Defect Memory 15

4.9. The UW-TIG Planner 15

4.10. Reporting Rule and Metrics 17

4.11. Re-implemented Baselines 17

4.12. Why PyBullet and Not Unity or Unreal 18

5. SYSTEM IMPLEMENTATION 19

5.1. Code Organisation 19

5.2. Implementing the Simulation 19

5.3. The Mission Loop 20

5.4. Experiment Harness and Protocol 20
5.5. Debugging Episodes 21
5.6. Demonstration and Comparison Videos 21

6. EXPERIMENTAL RESULTS 22

6.1. Detector Benchmark 22
6.2. How the Closed-Loop Numbers Reached 0.90 23
6.3. Closed-Loop Comparison and Ablations 24
6.4. Why Removing Uncertainty or Growth Changes Nothing 25
6.5. Head-to-Head with Published Planners 26
6.6. Calibration of the Uncertainty Signal 26
6.7. The Growing Scenario 27
6.8. A Broader Detector: Multi-Dataset Fine-Tuning 27
6.9. Scaling the Evaluation to a 31-Wall Yard 29
6.10. Does a Broader Detector Improve the Closed Loop? 31
6.11. Planner-Only Tests: Oracle Detector, Weight Sensitivity and Ten Seeds 31
6.12. Threats to Validity 32

7. FUTURE SCOPE AND ENHANCEMENTS 33

8. CONCLUSION 34

BIBLIOGRAPHY 35

ACKNOWLEDGEMENT

We would like to extend our sincere thanks to our Principal (FAC - In Charge), Dr. G. Thilagavathi, for providing us with the opportunity to develop and complete our project in our field of interest.

We express our sincere thanks to our respectful Head of the Department, Dr. G. R. Karpagam, for encouraging and allowing us to work on and present the project at our department for the partial fulfillment of the requirements leading to the award of the B.E. degree.

We take immense pleasure in conveying our thanks and a deep sense of gratitude to our faculty guide, Ms. L. Karthika, Department of Computer Science and Engineering, and our co-guide, Dr. N. Gopika Rani, for their relentless support, encouragement and suggestions, and for pushing us to test every claim against evidence rather than assumption throughout this project.

Finally, we thank our families and friends for their patience and support during the long experiment runs that this project required.

SYNOPSIS

Inspecting buildings, towers and pipelines for cracks, corrosion and leakage is repetitive work, and the autonomy that robots bring to it is usually limited to flying a fixed route or maximising how much surface is seen. Existing next-best-view planners decide where to look using geometry alone: they have no idea what a defect is, they forget everything between flights, and they never treat an unsure detection as a reason to look again. This project designs a planner that does.

Our contribution is **UW-TIG (Uncertainty-Weighted Temporal Information Gain)**. It keeps the cost-normalised information-gain formulation common to recent uncertainty-driven planners and adds a coverage guarantee, detection uncertainty from a trained YOLO detector, persistent cross-mission defect memory with growth tracking, a persistent-monitoring staleness term, and a two-step receding-horizon lookahead. Because a physical drone and a real defective structure are not needed to show or measure this, the second major contribution is a **reproducible simulation testbed**: an open-air utility yard in PyBullet textured with real photographs from the MBDD2025 dataset, a 54-viewpoint candidate graph (9 walls) that scales to a 186-viewpoint graph over 31 walls, four defect scenarios, a fast kinematic mode for statistics and a PID-flight mode for demonstration, and an evaluation harness with separate development and test seeds.

On held-out test seeds the original nine-wall yard gives UW-TIG a precision of 0.949, a recall of 0.967 and an F1 of 0.953, compared with five re-implemented published planners; it ties the strongest of them on detection quality (F1 0.950) while flying 39% less. To test whether this survives scale, the yard was enlarged to 31 inspectable walls and every planner re-run: UW-TIG reaches an F1 of 0.921 there, statistically level with the best classical rule and ahead of Rückin's IPP, with fewer metres flown than IPP. The detector was also fine-tuned on two further public datasets (dacl10k and a Roboflow corrosion set): its score on held-out dacl10k images rose from 0.004 to 0.184 mAP50 without changing its MBDD2025 score, but it did not improve the closed loop, so the original detector remains the default. The report also documents what did not work, what was wrong with our own measurements, and the limitations that remain.

LIST OF FIGURES

Fig. 4.1 System architecture: the five-stage closed loop 9

Fig. 4.2 Plan view of the yard: nine inspectable surfaces and the 54 candidate viewpoints 10

Fig. 4.3 Establishing shot of the simulated yard rendered with PyBullet 10

Fig. 4.4 Plan view of the 31-wall yard (buildings A–F, the pipe rack, the tower and six panels) and its 186 viewpoints 11

Fig. 4.5 UW-TIG decision flow for one step 15

Fig. 5.1 Same scenario, seed and timestep, three planners (left to right: UW-TIG, Isler-NBV, Random) . 20

Fig. 5.2 Onboard camera frame from the demonstration: real corrosion detections with confidence and uncertainty 20

Fig. 6.1 Per-class precision and recall of the default detector 21

Fig. 6.2 UW-TIG on development seeds as the fixes were applied 22

Fig. 6.3 Recall and flight distance of the ablations 23

Fig. 6.4 F1 and flight distance of all compared planners 24

Fig. 6.5 mAP50 of the original and fine-tuned detector on the MBDD2025 and dacl10k test sets 26

Fig. 6.6 F1 and flight distance of each planner in the 9-wall and the 31-wall yard (flight on a log axis) . 27

Fig. 6.7 Paired F1 difference between UW-TIG and each planner in the 31-wall yard, with 95% bootstrap intervals 28

LIST OF TABLES

Table 2.1 Summary of the reviewed planning literature 5
Table 3.1 Software stack 7
Table 3.2 Datasets used 8
Table 4.1 The four defect scenarios 13
Table 4.2 Re-implemented selection rules 17
Table 5.1 Repository layout 18
Table 6.1 Detector results on the held-out test set (1,448 images) 21
Table 6.2 The five fixes 22
Table 6.3 UW-TIG and its ablations on test seeds 0–4 (mean over static, uncertain, multi-defect; confirmed reports) 22
Table 6.4 Planner comparison on test seeds 0–4 (confirmed reports) 24
Table 6.5 Original and fine-tuned detector on three test sets 26
Table 6.6 Planners in the 31-wall yard, original detector (test seeds 0–4, confirmed reports) 27
Table 6.7 UW-TIG minus each planner, F1 per seed (31-wall yard, original detector) 28
Table 6.8 UW-TIG ablations in the 31-wall yard (original detector) 28
Table 6.9 UW-TIG with the original and the fine-tuned detector 29

1. INTRODUCTION

The deterioration of utility infrastructure – building facades, bridges, pipelines, transmission towers, etc. – is constant. The cracks open, the corrosion spreads and the leakage stains appear, and the only way to know is to take a close look, again and again, month after month, year after year. The rate of looking has become more affordable and safer, and flaws can now be identified in the photos unmanned aerial vehicles bring back with the help of deep-learning detectors. But more slowly has progressed the section between: deciding where the vehicle will head next.

In most deployed systems that decision is made prior to take-off. A route is designed, the drone follows it and then the pictures are analyzed afterward. A detection confidence does not influence the route, a defect detected in the previous month will not be remembered this month, and a blurry detection with low confidence will be trusted or discarded without being checked. Research on next-best-view (NBV) planning does fill the gap between what has been seen and where to go, but it optimises coverage of geometry: the planner is rewarded for exposing unseen surface whether or not that surface contains something of interest.

This project raises a simple question: What changes if the planner knows what is a defect, knows how certain the detector is and remembers what it saw the previous time? To answer it correctly requires two special attributes that normally aren't found in the same planner: 1) the planner needs to have those three abilities, and 2) an experimental environment in which the planner could be fairly compared to published alternatives, many times, with ground truth. This report pertains to both.

1.1. Objective

The objectives of the project are:

- In addition to the geometric information gain used by the planners of prior work (Rückin et al., 2023), it is necessary to design a UW-TIG planner which selects inspection viewpoints based on detector uncertainty, defect growth recorded in persistent memory and revisit staleness as well as a short lookahead.
- To create a simulation testbed where these and several published planners can be executed on the same scenes, detectors, seeds and scoring, and where the appearance of defects is real (photographs) and not synthetic (rendered).
- To measure the planner honestly: use development seeds to make every choice, report

  on separate test seeds, report every ablation of added mechanism, report failures as well as successes.

1.2. Overview

The system is closed loop of 5 stage (Chapter 4). Of the 54 possible viewpoints, a planner selects one. The view from the drone is simulated by the camera. A trained YOLO detector indicates defects with a certain confidence and uncertainty (which is the range of confidence over 5 copies of the frame, which are mostly perturbed in some way). The detection process is ray-cast from the camera to the known wall plane to get a 3-D location and a physical size. A persistent memory (PostgreSQL with pgvector, and Neo4j) correlates the detection to a defect it has previously encountered or creates a new one, and notes how the defect is expanding. The updated uncertainty and growth are fed into the planner's next decision making, and re-loaded beginning of the next mission.

The simulation is as much a part of the contribution as the planner. It is an open-air utility yard of nine inspectable surfaces with textured from true MBDD2025 photographs, a candidate-viewpoint graph with two stand-off distances selected from a detector diagnostic, four scenarios (static, uncertain, growing, multiple defects) and an evaluation protocol of a development/test split. The simulation is inexpensive which allowed the planner comparison to be conducted over the 5 test seeds × 4 scenarios × 12 planners × 3 missions, which would have been prohibitively expensive in the field with a real drone and is additionally not repeatable exactly in the field.

1.3. Scope

In scope: the UW-TIG planner and its ablations; re-implementations (at the level of the viewpoint-selection rule) of four published planners; the PyBullet testbed; a YOLO detector trained on a public dataset; the memory and geometry components that connect them; and a quantitative comparison with a statistical protocol.

Out of scope: hardware flight; obstacle avoidance and dynamics beyond a PID-controlled quadrotor model; photo-realistic rendering (see Section 4.12 for an explanation why); and the authors' full original systems for the compared papers. These boundaries are reiterated in Chapters 6 and 7, along with the consequences.

2. LITERATURE SURVEY

2.1. Literature Survey

2.1.1. Uncertainty - aware informative path planning

Rückin et al. [1] use a model's epistemic uncertainty (MC-Dropout) to determine where to look, normalised by the travel cost, to plan UAV paths for active learning in the context of semantic mapping. The structure is the most similar to UW-TIG with regards to its utility, but this uncertainty is used in one session to improve the model rather than to track a set of fixed defects over sessions. Vashisth et al. [2] propose to learn adaptive informative path planning on dynamic graphs using deep reinforcement learning, whereas Moon et al (IA-TIGRIS) [3] propose a sampling based planner that reuses planning effort and adapts to updated belief maps for both multirotor and fixed-wing UAVs, and Li et al. [4] propose an algorithm for selecting views under motion and measurement uncertainty for reconstructing moving objects. These works have seen the field move towards learned and adaptive planners but all of them reason about information gain more over a generic belief than defects with an identity and a history.

2.1.2. Inspection-specific planning

GATSBI [8, 9] was introduced by Dhami et al. for an online algorithm which routes a UAV over generalise a travelling-salesman problem to target surfaces of a bridge; and A defect detector is added to extension. It is the most similar system to inspection-aware that has been published. Neither has a detection-uncertainty term nor does it plan in more than one session. memory across sessions. Pred-NBV and MAP-NBV [10] are approaches based on shape completion, which are learned. Modelling as a means to guide views, is an indirect uncertainty about geometry, not about. defects.

Petit and Lussier Desbiens [5] (MOAR) plan risk-aware, multi-objective paths for infrastructure inspection that adapt to weather and battery state; Svystun et al. [6] adapt trajectories to the pitch of wind-turbine blades using segmentation, reporting a 78% reduction in inspection time; Jin et al. [7], extended as ASIP-Planner [26], combine global coverage planning with local reactive re-planning for surface inspection in partially unknown indoor environments; FC-Vision [27] repairs viewpoints in real time when unexpected obstacles occlude a scan. None of these plans around detected defects, and none revisits a defect across missions. Two reviews [11, 12] of decision-making UAV path planning and of UAV infrastructure inspection both identify perception-driven flight planning, and fusion of detection with planning, as an open problem.

2.1.3. Classical building blocks

The recent planners above rest on older ideas, three of which UW-TIG also uses as ingredients. Isler et al. [23] formulate next-best-view selection as maximising an

entropy-based information gain, normalised by travel cost; Bircher et al. [24] plan over a tree of motions and execute only the first step (a receding horizon); Alamdari, Fata and Smith [25] minimise the maximum latency between observations for persistent monitoring. We re-implement these three selection rules as baselines so that the comparison spans both generations of planner, but they are not the reference point of this work: they have no notion of a defect, and the contribution is measured primarily against the recent, perception-aware planners of Sections 2.1.1 and 2.1.2.

2.1.4. Defect detection and datasets

Zha et al. [13] publish MBDD2025, 14,471 UAV photographs of building surfaces in six structure types and five defect classes (crack, leakage, abscission, corrosion, bulge), with bounding-box annotations and a benchmark of 35 detectors; it is the dataset used here. Recent detectors report high scores on narrower problems: Huang et al. [14] reach 0.913 precision, 0.766 recall and 0.864 mAP@50 for concrete cracks with an improved YOLOv11; Inam et al.

[15] report 0.977 / 0.967 / 0.993 for bridge cracks (one class); Li, Shi and Sun [16] report 0.932 / 0.924 / 0.926 on a private 5,000-image utility-tunnel dataset. Shen et al. [17] proposed a transformer detector for detecting defects in transmission lines, and Hebbache et al. [18] proposed a saliency-guided YOLOX detector for bridge defect detection. These results are not directly comparable to a five-class public benchmark, a point Chapter 6 returns to. For bridges, Flotzinger et al. [28] released dacl10k (9,920 inspection photographs, 19 classes, segmentation-first), and its follow-up [29] shows that rebalancing it with synthetic crack and cavity textures improves segmentation robustness by about two points; Roboflow-100 [30] is a public multi-domain detection benchmark from which a corrosion set is taken here; and Özgenel and Sorguç [31] and Dorafshan et al. [32] (SDNET2018) are the standard concrete-crack image collections.

2.1.5. Simulation platforms

AirSim [19] builds on Unreal Engine for high-fidelity rendering and physics; Flightmare [20] couples a Unity renderer with a separate fast physics engine; GRaD-Nav [21] represents the frontier with 3-D Gaussian Splatting. All three of these cover the need for bridging the gap between. When using synthetic images for model training, rendered and real imagery when gym-pybullet-drones.The PID controlled Pybullet quadrotor with an onboard camera as described in [22] is the physics layer used here.

Table 2.1 The literature reviewed has been summarized.

Paper and authors	Technique	Advantages	Limitations
Rückin et al. [1]	MC-Dropout uncertainty drives path	Real model uncertainty in planning	Single session; goal is retraining
Dhami et al. (GATSBI) [8,9]	GTSP routing + detector	Inspection-aware, cost-aware	Single session; no uncertainty/memory
Vashisth et al. [2]	Deep RL on dynamic graphs	Learned, adaptive	Needs training; no defect history
Petit & Lussier Desbiens [5]	Risk-aware multi-objective planning	Weather/battery adaptive	No perception or defects
Svystun et al. [6]	Segmentation-guided trajectories	78% shorter inspection	One structure type; no revisits
Jin et al. [7]	Global coverage + local replanning	Handles unmapped obstacles	Coverage only
Moon et al. [3]	Incremental sampling-based IPP	Fast online replanning	Generic gain; no defects
Li et al. [4]	NBV under motion uncertainty	Plans over uncertain futures	Reconstruction, not inspection
Isler / Bircher [23,24]	Entropy-gain NBV; receding horizon	Classical building blocks (baselines)	Geometry only; no defects, no memory
Alamdari et al. [25]	Max-latency revisit scheduling	Guaranteed regular revisits	Time only; no perception

2.2. Inference

There are three patterns revealed in the survey. Having become more capable, first, planning has become more... recent planners add learning, risk-awareness and incremental to not more defect-aware re-planning, but the goal is to provide coverage/information gain to a generic belief. Secondly, there is uncertainty-driven planning that takes place within one planning session and that is directed at at improving a model, NOT at monitoring defects.

Detection: The best published figures are on the single-class or neither private data, nor the planning literature, ever measures against a real trained detector, Instead, however, they used assumed ground-truth detections. A planner evaluated against a real detector inherits its failure modes – low recall at short range, false positives from neighbouring surfaces, poor calibration – and these turned out to matter greatly in this project (Chapters 5 and 6).

2.3. Research Gap

No reviewed work combines, in one planner that is evaluated against a real trained detector: (i) detection uncertainty as a reason to revisit; (ii) defect identity and growth remembered across missions; (iii) revisit staleness; and (iv) a cost-aware, short-horizon decision rule. Furthermore, there has been no reviewed work which has offered a controlled testing bed for such a phenomenon. planner can be compared with several published planners under identical conditions, with real defect imagery and a development/test split.

2.4. Research Objectives

- Introduce UW-TIG, a planner whose utility is a combination of coverage information gain, The two-step detector uncertainty and defect growth are selected, to cover the staleness and cost. With an explicit coverage guarantee, a lookahead algorithm.
- All planners are judged on the same scenarios and a leakage-free memory, so that same inputs.
- Re-practice four published planners' selection rules and compare results with these rules. UW-TIG and with random search.
- Eliminate all added mechanisms and report if any of them measurably help, if none, if they help somewhat. and why.
- Limitations, such as calibration of the uncertainty signal, and failure to track, should be reported. the growing-defect scenario.

3. SYSTEM REQUIREMENTS

3.1. Hardware Requirements

- Development machine: One NVIDIA L4 (24 GB) GPU on a rented 32-vCPU machine.
- Memory: several GB are required to store the dataset (MBDD2025, 14,471 images and labels), the model weights, results, and logs.
- GPU: One NVIDIA L4 (24 GB) GPU is used for the 100-epoch training of the YOLOv8s model, which is performed once to train the entire sweep of the model.
- This is followed by a retrain of the YOLOv8s model with the later YOLO11m model, where the same model is trained with a different set of images to build additional data.
- This is then followed by fine-tuning the final model on a merged set of images, and finally, parallel sweeps of the planner on the 31-wall yard are run using the 100-epoch YOLOv8s model.
- The initial nine wall sweep was installed on the Windows workstation.

3.2. Software Requirements

Table 3.1 Software stack

Component	Tool / version	Purpose
Language	Python 3.12	All simulation, planning, perception and analysis code
Physics and rendering	PyBullet; gym-pybullet-drones	Quadrotor with PID control (DSLPIDControl), onboard camera, TinyRenderer
Detection	Ultralytics YOLOv8 / YOLO11; OpenCV; NumPy	Defect detector, test-time augmentation, image resampling
Persistent memory	PostgreSQL + pgvector; Neo4j	Detection log, appearance/spatial matching; defect identity graph
Deployment	Docker Compose	Runs both databases locally
Textures	Pillow	Wall, ground and sky texture generation and cropping
Analysis and reporting	matplotlib; ReportLab; pptxgenjs	Charts, this report, the review deck
Version control	Git / GitHub	Reproducibility of every reported number

3.3. Datasets

Four public datasets are used, each for a different purpose; a fifth, SDNET2018, was planned but could not be obtained. Table 3.2 lists them. MBDD2025 provides both the detector's training data and, as wall textures, the defect appearance in the simulation. The two further datasets were added to test whether a broader training set improves the detector, and the concrete-crack crops provide an image-level check on data never trained on.

Table 3.2 Datasets used

Dataset	Content	Use here	Size used
MBDD2025 [13]	UAV photographs of building surfaces; 5 classes (crack, leakage, abscission, corrosion, bulge); boxes	Detector training, validation and test; wall textures	10,129 / 2,894 / 1,448 images (train / val / test); training adds 2,826 bulge duplicates
dacl10k [28]	Real bridge-inspection photographs; 19 classes with polygon and box labels (CC-BY-4.0)	Fine-tuning (train split); held-out cross-dataset test (validation split)	5,362 train + 466 val; 810 held out
Roboflow-100 corrosion [30]	Corrosion and crack boxes (CC-BY-4.0)	Fine-tuning	851 images
Concrete cracks (Özgenel) [31]	227 × 227 crops, cracked or not (CC-BY-4.0)	Image-level crack-presence check; never trained on	400 cracked + 400 uncracked
SDNET2018 [32]	56,000 concrete crack / no-crack crops	Planned; download blocked by a bot check on every mirror we reached	not used

Class mapping. There are 19 classes in dacl10k and five in MBDD2025. The classes bulge, other classes, do not have a counterpart in dacl10k; the classes that are counterparts in dacl10k are: crack and crack, wet spot and leakage, efflorescence and leakage, spalling and abscission, rock pocket and abscission, rust and corrosion, exposed rebar and corrosion. This mapping is a judgement; it's not a fact – in particular, efflorescence is a water-related deposit, not a leak, hence the leakage labels are noisier. The Roboflow set maps the corrosion and crack directly and drops its "Slippage" class, which were not used as the polygon masks of the dataset.

Held-out discipline. The validation split of MBDD2025 is not used when training and early stopping, while 8% of the training split is reserved for validation.

4. SYSTEM DESIGN

4.1. Overall Architecture

The closed loop is shown in Fig. 4.1. The planner and the simulated drone and camera, the detector, the geometry module and persistent memory, make a cycle, and the feedback from memory to planner, is the difference between this system and a one-shot pipeline. There are two modes of operation for the same loop (Section 4.3): fast kinematic mode for statistics; PID-flight mode for demonstration.

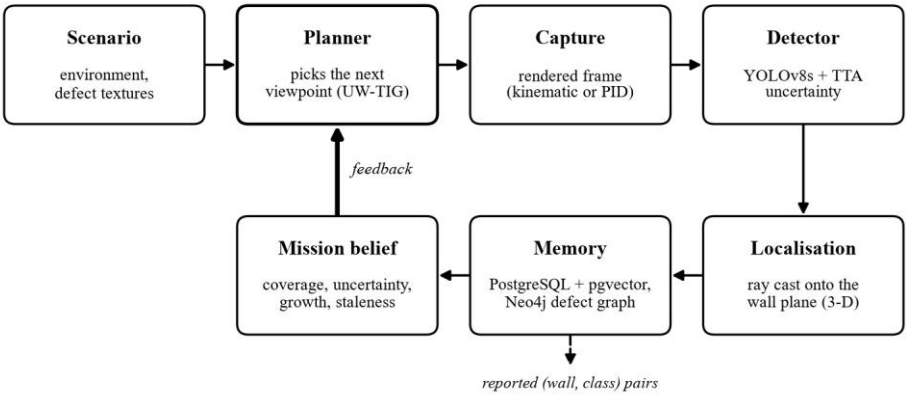


Fig. 4.1 System architecture: the five-stage closed loop

4.2. Simulation Environment

The environment is an open-air utility yard (Fig. 4.2). It supplanted a previous enclosed two-room house: In an enclosed building the drone does not encounter anything other than walls, the sky is a white abstraction and the scene is a tour of the house and not infrastructure. The yard has two freestanding buildings – Building A (four walls, 4 m × 4 m) and Building B (three walls, 2.6 m × 2.6 m) – about 11 m apart with open sky between them, and one inspection panel each on a decorative pipe rack and a lattice tower. In total there are nine inspectable surfaces. A sky backdrop with a skyline silhouette, a procedurally generated concrete ground texture and small props (crates, barrels) dress the scene.

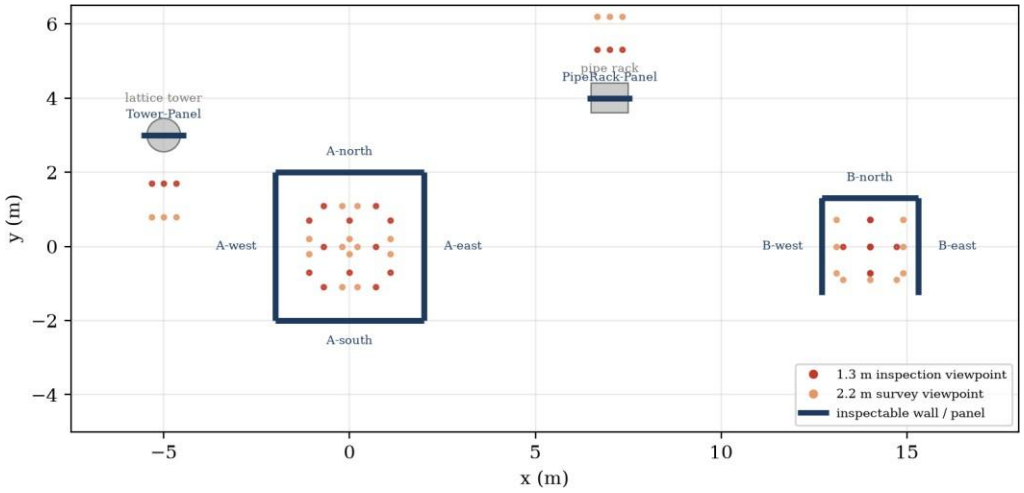


Fig. 4.2 Plan view of the yard: nine inspectable surfaces and the 54 candidate viewpoints

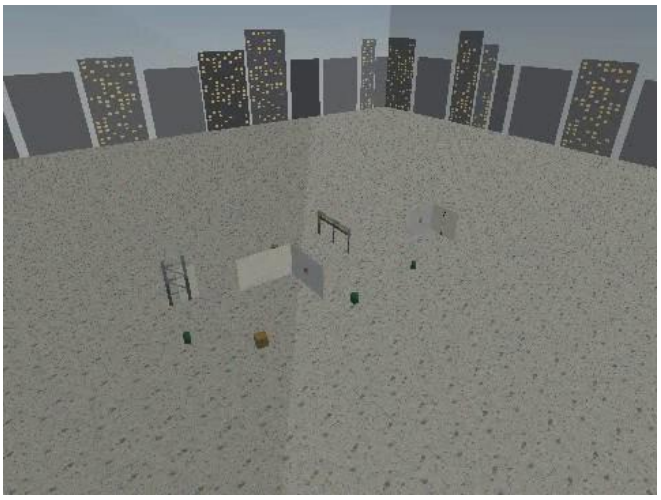


Fig. 4.3 Establishing shot of the simulated yard rendered with PyBullet

4.2.0. Two layouts: nine walls and thirty-one

Every result up to Section 6.7 was produced in the nine-wall yard described above. A fair objection is that nine walls and a 16-step budget are an easy problem: a planner that visits every wall once has nearly solved it. The yard was therefore enlarged (selected with the environment variable UWTIG_YARD=large) to 31 inspectable surfaces: four more equipment buildings (C 4 m × 4 m, D and E 3.2 m × 3.2 m, F 2.6 m × 2.6 m) and six more freestanding inspection panels, spread over roughly 34 m × 28 m. The candidate graph grows from 54 to 186 viewpoints (six per wall) and the mission budget from 16 to 50 steps, keeping the ratio of budget to walls near 1.7. Fig. 4.4 shows the layout.

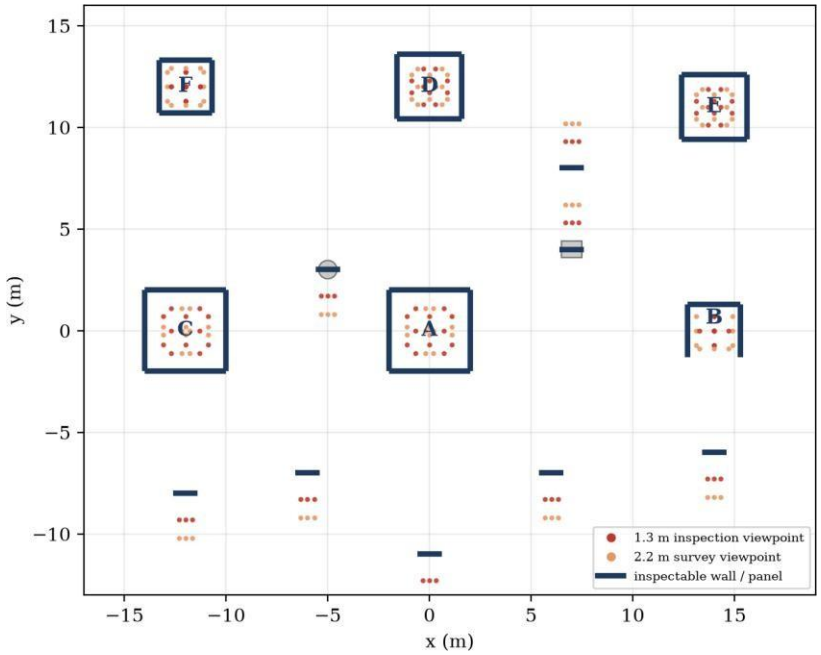


Fig. 4.4 Plan view of the 31-wall yard (buildings A–F, the pipe rack, the tower and six panels) and its 186 viewpoints

4.2.1. Walls as textured quads

Every inspectable surface is a flat quadrilateral mesh with a texture bound to it. A wall is defined by its centre, its yaw and its width; the plane it lies on, its outward normal and its in-plane axes follow from these (geometry.wall_plane). Using flat quads rather than boxes keeps the geometry of every wall analytically known, which is what makes ray-casting a detection back onto the wall exact (Section 4.7).

4.2.2. Real defect imagery as wall texture

A wall's defect is a real photograph from MBDD2025 applied as its texture. For a defect scenario, a photograph containing the wanted class is selected and a region is cropped to the wall's true aspect ratio (width divided by the 2.4 m wall height) at a long side of 1536 pixels. This crop was not in the first design: the early textures were square images stretched over walls up to 4 m wide, which distorted defects by roughly 2.2× and left them pixelated, so the detector was being tested on imagery it had never been trained for. Ground-truth defect centres are kept in a fixed 512-unit texture space so that the same physical spot is reproduced from mission to mission and can be converted to a world position analytically (texture_uv_to_world).

Real photographs also contain defects of other classes: about 16% of MBDD2025 photos carry a second labelled defect. The scenario therefore records *which classes are present* in the cropped region, so that detecting a second real defect is not scored as a false positive (Section 4.10).

4.2.3. Camera and rendering

The onboard camera follows the gym-pybullet-drones convention: a 60° field of view rendered into a 320 × 240 frame with the projection matrix built for an aspect ratio of 1.0. The frame is therefore stretched horizontally by 4/3 relative to a square-pixel image. Because the detector was trained on undistorted photographs, every frame is resampled to square pixels before inference and the detected boxes are mapped back, so that localisation still uses the original geometry (Section 4.6).

Rendering uses PyBullet's TinyRenderer, a software rasteriser with shadow mapping. It is not a physically based renderer; the realism work in this project was limited to what changes the *detector's* input: consistent light direction and shadows, higher-resolution ground and sky textures, and a fix for a texture-tiling bug in box geometry (Section 5.2).

4.3. Two Simulation Modes

Detection and planning quality do not depend on rotor dynamics, so the statistical comparison does not need to step a physics engine for every flight. The testbed therefore has two modes that share the same scene, the same camera convention and the same planner code:

- Kinematic mode (experiments/kinematic_capture.py): the virtual camera is teleported to a candidate viewpoint and rendered. No PID stepping is performed. This mode runs the planner comparison.
- PID-flight mode (demo_uwtig_flight.py): a gym-pybullet-drones quadrotor flies between viewpoints under DSLPIDControl, with a smoothstep (ease-in, ease-out) reference to avoid velocity discontinuities, and a smoothed chase camera. This mode produces the demonstration videos.

The two modes use the same localisation mathematics, so what is verified in one carries to the other. The demonstration script accepts any planner by name, which allows the same scenario, seed and budget to be flown by different planners and the videos placed side by side – a way of *showing* the difference without any physical drone.

4.4. Candidate Viewpoint Graph

The viewpoint generator generates 54 viewpoints (2 standoff distances (1.3 m and 2.2 m) and 3 lateral offsets (−0.55, 0 and +0.55) for each of the 3 walls. Every viewpoint has its pose and the cells it can see in its wall. Surface coverage is discretised into cells of a fixed physical width of 1 m, so a 4 m wall has four cells and a 1.2 m panel one. This choice, following the fixed voxel *size* of classical volumetric planners [23], fixed a failure found early: with a fixed number of cells per wall, a narrow wall's single viewpoint saw all of its surface at once, earned a disproportionate

information gain, and the planner froze on the small panels for an entire mission.

The 2.2 m standoff was added after a detection diagnostic on development seeds (Section 6.2). The cost of moving between viewpoints is the Euclidean distance plus a very small rotation term (1/3600 m per degree of yaw), whose only job is to break ties between viewpoints at exactly the same position, which occur in Building B because its 2.6 m width is exactly twice the 1.3 m standoff.

4.5. Scenarios

Table 4.1 The four defect scenarios

Scenario	Defects	Severity	Purpose
Static	A-west crack, A-east corrosion, B-north leakage (MBDD2025 photos)	1.0 (fixed)	Steady-state detection and coverage
Uncertain	A-south crack, B-east corrosion (MBDD2025 photos)	0.35 (faint)	Low-confidence, ambiguous defects
Multi-defect	Six walls, three classes (MBDD2025 photos)	0.9	Coverage breadth vs. reinspection depth
Growing	A-east corrosion, procedurally generated	0.4 to 1.3 over missions	Known-growth ground truth for the memory

Three scenarios use real photographs. The fourth, *growing*, needs a defect whose true size changes by a known amount between missions; no single-time-point photograph dataset can provide that, so it uses a procedural texture. This is the one scenario where the detector fails, for a reason documented in Section 6.7. In the 31-wall yard the defect walls are not a fixed list: each scenario draws its walls without replacement from all 31 by a random generator seeded with the scenario and seed (8 walls for static, 6 for uncertain, 2 for growing, 16 for multi-defect, classes cycled), so defect placement varies from seed to seed.

4.6. Perception

The detector is a YOLOv8s model at 640 px trained on MBDD2025 with a 70/20/10 split (Section 6.1). For each frame the system builds five inputs: the square-pixel frame and four photometric variants (brightness/contrast, additive noise, blur, and a blurred, re-brightened copy) that move no pixel geometry. The model runs on all five. For every detection in the original frame, the confidence of the best-overlapping same-class box in each variant (zero if there is none) is collected, and the standard deviation across the five is reported as the detection’s uncertainty. This test-time-augmentation ensemble is a substitute for MC-Dropout; Section 6.6 and Chapter 7 discuss why a literal MC-Dropout was not adopted.

4.7. 3-D Localisation

A detection's centre pixel is turned into a ray using the camera's view and projection matrices and intersected with the plane of the wall being inspected. A hit is accepted only if it lies inside the wall's physical rectangle (with a 5 cm margin). The rectangle test matters: a ray through a pixel that shows an adjacent wall, the ground or a prop still intersects the inspected wall's *infinite plane* somewhere, so without the test the defect was attributed to the wrong surface. On development seeds this accounted for 51–75% of all false positives, against one of 1,369 true positives. The box's two corners are localised the same way (without the rectangle test) to give a physical size in metres, used for growth.

4.8. Persistent Defect Memory

Memory has two stores. PostgreSQL with pgvector holds an append-only log of every detection and uses the stored appearance embedding of the cropped detection and its world position to match a new detection to an existing defect. Neo4j holds the defect graph: defect nodes linked to the missions that observed them and the walls they lie on. When a detection matches a known defect, its *growth* is the difference between its current and previous physical size; otherwise a new defect is registered. At the start of every mission after the first, the planner's belief is seeded with each active defect's last uncertainty and its growth. Memory keys are namespaced per scenario, planner and seed, so that no planner can see another's detections (Section 5.4). The 31-wall sweeps ran on a rented machine without Docker, so a second, in-process implementation of the same store was written (memory/store_mem.py). It ports the matching rule exactly – among the five nearest detections of the same class on the same wall by cosine distance of the embedding, the nearest one within 0.35 m is the same defect – but does not mirror the Neo4j graph, which no planner or metric reads. The two implementations were not run against each other on the same sweep, so this is a faithful port, not a verified equivalent.

4.9. The UW-TIG Planner

This section is the core of the contribution. The planner chooses the next viewpoint v from the 54 candidates. Its design has one front-end rule and one utility.

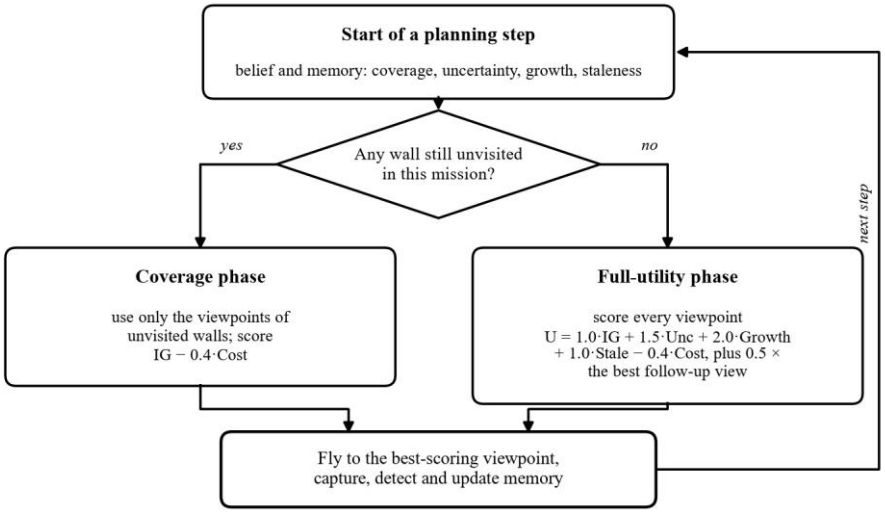


Fig. 4.5 UW-TIG decision flow for one step

4.9.1. The utility

$$\mathbf{U(v) = w_{ig} \cdot IG(v) + w_{u} \cdot Unc(v) + w_{g} \cdot Growth(v) + w_{s} \cdot Stale(v) - w_{c} \cdot Cost(v)}$$

with $w_{ig} = 1.0$, $w_u = 1.5$, $w_g = 2.0$, $w_s = 1.0$ and $w_c = 0.4$. The terms are as follows.

- Information gain IG(v) (the cost-normalised entropy term of [23], shared by the planners of Chapter 2). Each cell has a belief $p = 1 - 0.5 \cdot e^{-0.6 \cdot n}$ that it is already adequately observed after *n* views; its entropy is highest (1 bit) before any view and falls with each view. IG(v) is the sum of entropies of the cells the viewpoint can see.
- Uncertainty Unc(v). For each visible cell, the uncertainty last recorded there by the detector (Section 4.6), faded by $e^{-0.6 \cdot n}$ so that re-viewing the same cell earns diminishing reward. This is what makes an unsure detection worth a second look.
- Growth Growth(v). The non-negative growth recorded in memory for the defects in view, with the same fading. This is the temporal term that lets a worsening defect attract the drone.
- Staleness Stale(v). For every cell, if it has not been seen for the past 15 steps, its attractiveness increases by $1 - e^{-0.15 \cdot t}$, where t is the number of steps elapsed since the last time the cell was seen (after the maximally-latent idea of Alamdari et al.).
- Cost(v). The small rotation term from Section 4.4 and the distance from the present point of view.

4.9.2. The coverage guarantee

While any wall has not been visited *in this mission*, the planner restricts itself to the viewpoints of the unvisited walls and scores them with the plain information-gain rule alone, IG(v) − 0.4·Cost(v). Once every wall has been seen once, the full utility takes over. This was

added after a surprising observation: Isler-NBV and every early UW-TIG variant stopped at exactly 4 of 9 walls on every seed. Since a planner with *none* of the new terms hit the same ceiling, no re-weighting of the new terms could fix it. In the cost-normalised greedy rule, flying to the far building is never worth its cost against the information gain still available nearby. A weight is a fragile thing to hang a guarantee on; an explicit phase is not.

4.9.3. Two-step lookahead

In the full-utility phase, each candidate v_1 is scored as $U(v_1) + 0.5 \cdot \max_{v2} U(v_2 \mid \text{after } v_1)$, where the second step uses the view counts that would result from visiting v_1 but the current uncertainty, growth and staleness (a future detection cannot be simulated). Only the first step is executed, after which the planner re-plans, in the manner of a receding-horizon planner [24].

4.9.4. Why the terms fade with repeated views

An early version let uncertainty and growth keep their full reward on every revisit. A real, persistent defect is re-detected on every look, so once found it earned its full reward again at no extra travel cost, and the lookahead exploited this so completely that flight distance collapsed to exactly 0.0 m for every UW-TIG variant: the planner camped on one viewpoint. Fading the terms with the view count, the same saturation used for coverage entropy, removed the exploit.

4.10. Reporting Rule and Metrics

A planner's output for a mission is the set of (wall, class) pairs it *reports*. A pair is reported once it has been seen in at least three separate looks (counted across missions, since memory carries the count) or once at confidence ≥ 0.45; the rule is identical for every planner and was chosen on development seeds by planner-averaged F1. Precision, recall and F1 are computed over these reports against the scenario's ground truth, with classes that are really present in the photograph but are not the scenario's primary class *ignored* (as in COCO-style evaluation). Other metrics are mean localisation error (metres), total flight distance, wall coverage, reinspection rate, expected calibration error (ECE) of the detector's confidence, and an uncertainty gap (mean uncertainty on false positives minus on true positives). A "legacy" precision that divides unique true positives by every raw false-positive box is also kept, to show how much of the improvement is a change in measurement (Section 6.2).

4.11. Re-implemented Baselines

Four published planners were added to the registry, each adapting the paper's viewpoint-selection rule to the same viewpoint graph and belief. They are faithful to the *rule*, not to the authors' full systems. Their parameters were set once from the papers' formulations and not tuned on any result.

Table 4.2 Re-implemented selection rules

Registry name	Paper	Selection rule
bircher_rhnbv	Bircher et al.	Best depth-2 branch of gain × $e^{-0.5 \cdot \text{cost}}$; execute the first step only
gatsbi_gtsp	Dhami et al. (GATSBI)	Nearest-neighbour + 2-opt tour over the least-inspected walls, followed in order
ruckin_ipp	Rückin et al.	Acquisition Σ(uncertainty, 0.5 if unobserved) / (1 + visit count), single-session belief
alamdari_latency	Alamdari et al.	Greedy on maximum time since last visit
isler_nbv / random	Isler et al. / none	Cost-normalised entropy gain over unvisited views / uniform random

4.12. Why PyBullet and Not Unity or Unreal

One possible question is whether using a natural game-engine or Gaussian-splatting renderer will make the simulation more believable. According to a literature check, the answer to this project is no. To bridge the gap between rendered and real imagery when a model is trained on rendered images, there are tools such as AirSim [19], Flightmare [20] and GRaD-Nav [21]. In this scenario, the camera remains fixed on a wall to be inspected and the detector is trained and tested only on real photos, as textures. A prettier renderer would make a prettier game to play, but wouldn't change any one pixel the detector was taught on. PyBullet was thus retained, as it can be scripted from a terminal, can be run in parallel, and gym-pybullet-drones already provides PID flight. Section 6.8 specifies the honest limits, which include a software rasteriser, no sensor noise model and no wind.

5. SYSTEM IMPLEMENTATION

5.1. Code Organisation

Table 5.1 Repository layout

Module	Responsibility
sim_house.py; scene/environment.py	Wall layout, textured quads, ground, sky, pipe rack, tower, props
experiments/kinematic_capture.py	Teleport-and-render camera for the statistical sweeps
demo_uwtig_flight.py; make_comparison_video.py	PID-flight demo; side-by-side planner videos
datagen/mbdd_textures.py; prepare_mbdd.py; oversample_bulge.py	MBDD2025 preparation, aspect-correct wall textures, class oversampling
perception/ml_detector.py; train_yolo.py; train_yolo_gpu.py	Detector with TTA uncertainty; CPU and GPU training
geometry.py	Camera matrices, pixel rays, wall planes, on-wall localisation
memory/store.py	PostgreSQL/pgvector log, Neo4j graph, identity matching and growth
planning/viewpoints.py; planning/planners.py	Viewpoint graph and cells; belief, UW-TIG, ablations and baselines
experiments/scenarios.py; mission.py; evaluate.py	Scenarios; the mission loop; scoring and the sweep driver
experiments/diagnose_detection.py; tune_report_rule.py	Planner-independent detector diagnostic; report-rule tuning on dev seeds

5.2. Implementing the Simulation

Walls. A single quad mesh (scene/wall_quad.obj) is scaled and rotated per wall, with its UV mapping chosen so that a texture pixel maps to a known world point; this analytic inverse was validated against the ray-plane path to under 2 cm for a known marker, and is used to score localisation error. Lighting. The capture call uses a fixed light direction with shadows enabled so that shading is consistent from view to view. Textures. Ground (concrete) and sky (gradient with a skyline silhouette) are generated procedurally at higher resolution than the first version; the sky is a backdrop around the yard so that the horizon is no longer a blank white void.

A tiling bug. A texture-tiling bug in PyBullet's box geometry (GEOM_BOX) was found during the realism pass and fixed. A resolution hypothesis that was wrong. Because early recall was low, the first suspicion was image resolution. A higher camera and inference resolution was tried and *reduced* recall (0.42 to 0.22). The causes turned out to be texture distortion and viewing distance (Section 6.2), which would not have been found if the hypothesis had been accepted without measurement.

5.3. The Mission Loop

Algorithm 1 summarises one mission (experiments/mission.py). The scenario sets the wall textures for this mission index and supplies ground truth; the belief is seeded from memory if the mission index exceeds one; then, for each of the 16 steps, the planner chooses a viewpoint, the camera captures it, the detector runs, and each detection is localised, matched in memory, and written back into the belief.

```
Algorithm 1: one mission
  belief <- MissionBelief(walls)
  if mission > 1: belief.seed_from_memory(run_key)
  scenario.setup_mission(mission) # textures + ground truth
  for step in 1..16:
      v    <- planner.select_next(belief, current)
      img  <- camera.capture(v)
      for d in detector(img):                # class, confidence, uncertainty
          xyz  <- localize_on_wall(d.center, pose, wall) # reject if off-wall
          size <- distance(localize(d.corner1), localize(d.corner2))
          id, growth <- memory.match_or_create(run_key, wall, d, xyz, size)
          belief.record_detection(wall, xyz, d.uncertainty, growth)
      belief.record_visit(v); current <- v
```

5.4. Experiment Harness and Protocol

- Development / test split. Every design choice (viewpoint standoffs, reporting rule, fixes) was made on development seeds 100–102. The numbers reported are from one run on test seeds 0–4 that was not tuned on
- Sweep size. 4 scenarios × 12 planners × 5 seeds × 3 missions, 16 viewpoints per mission; results are averaged over static, uncertain and multi-defect (growing is reported separately).
- Isolation. Memory keys are namespaced as scenario|planner|seed. An early sweep let the planners' detection histories leak into each other because identity matching was not scoped; a targeted test caught this, and it was fixed before any reported number was produced.
- planners' detection histories are not crossed-checked so they overlap.
- Process isolation. A single long process running about 140 mission-runs crashed three times with three different symptoms (allocation failure, silent exit, segmentation fault), consistent with PyBullet's software renderer accumulating resources over many render sessions. Each seed now runs in its own subprocess and the per-seed CSVs are merged (run_full_sweep.sh).
- Parallel sweeps. For the 31-wall yard each (seed, scenario) pair is a separate subprocess, and fourteen run at once (experiments/run_parallel_sweep.py); the parts are concatenated into a single CSV. Each large sweep is 240 rows and took about 30 minutes.

5.5. Debugging Episodes

Four defects in the work itself shaped the final design, and are recorded here because each changed a conclusion.

- Camping. The lookahead exploited non-decaying uncertainty/growth rewards (flight distance 0.0 m). Fixed by fading the terms with view count (Section 4.9.4).
- Free spinning. Building B's coincident viewpoints made turning in place free. An over-large rotation cost fixed it but collapsed Isler-NBV's coverage from 0.875 to 0.333; the final constant (1/3600 m per degree) breaks only exact ties.
- Cells. A fixed cell count per wall froze the planner on narrow panels; fixed physical cell width fixed it (Section 4.4).
- Wrong-surface hits. Ray hits outside the wall's rectangle were blamed on the inspected wall (Section 4.7).

5.6. Demonstration and Comparison Videos

demo_uwtig_flight.py flies a chosen planner under real PID control through three missions of the multi-defect scenario, writing a third-person flythrough and an onboard-camera video with detections, confidences and uncertainties overlaid. make_comparison_video.py splices the flythrough or inspection videos of different planners frame-by-frame, because the step count depends on the budget and mission count, not on the planner. Fig. 5.1 is one such frame.



Fig. 5.1 Same scenario, seed and timestep, three planners (left to right: UW-TIG, Isler-NBV, Random)

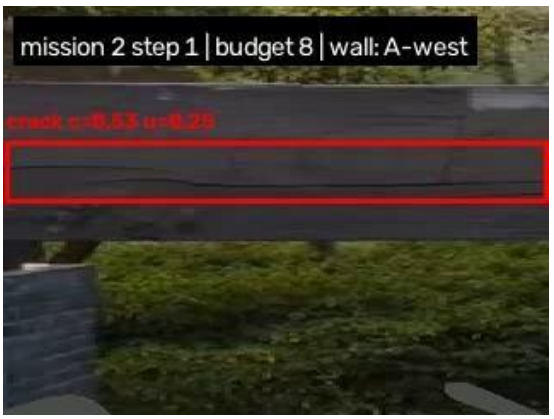


Fig. 5.2 Onboard camera frame from the demonstration: real corrosion detections with confidence and uncertainty

6. EXPERIMENTAL RESULTS

6.1. Detector Benchmark

The detector is evaluated on the held-out MBDD2025 test split of 1,448 images. Two models were trained with identical settings (640 px, up to 100 epochs, the same L4 GPU and the same data): the default YOLOv8s, with the rarest class, bulge, oversampled about four times, and a larger YOLO11m retrain, which was evaluated but not adopted. An earlier YOLOv8n run (320 px, 30 epochs, CPU) was only a smoke test of the pipeline; its resolution, epochs and hardware differ, so it is not compared here. Larger YOLOv8 variants (m and l) were not trained.

Table 6.1 Detector results on the held-out test set (1,448 images)

Model	mAP50	mAP50-95	Precision	Recall
YOLOv8s, 640 px, 100 epochs (default)	0.884	0.506	0.874	0.842
YOLO11m, 640 px (retrain; not adopted)	0.890	0.510	0.869	0.863

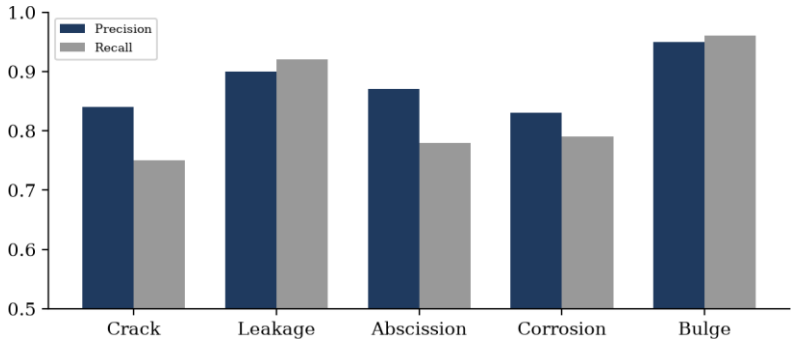


Fig. 6.1 Per-class precision and recall of the default detector

Crack is the hardest class (recall 0.75): cracks are thin, low-contrast features that lose most of their signal at low resolution, which is why moving from 320 px to 640 px nearly doubled crack recall. Bulge, the best class (0.95 / 0.96), was the rarest in the data and was oversampled. The default detector does *not* reach 0.90 on mean precision or recall; all the above-0.90 numbers in this report are closed-loop report-level metrics (Section 6.3), not detector metrics. Compared with the literature, this detector is below the single-class and private-data scores of Section 2.1.4 and above the recall of Huang et al.'s crack detector (0.766) on a harder five-class public task.

The rows above differ in resolution, epochs and data, so they are not a fair comparison of model size. YOLOv8n, v8s and v8m were therefore retrained with exactly the same recipe: the same fixed random subset of 4,000 MBDD2025 training images, 640 px, 30 epochs, batch 8, seed 0 and no oversampling, on one RTX 5060 GPU (datagen/compare_detectors.py). All three were scored on the full 1,448-image test split. YOLOv8l was not trained (8 GB of GPU

memory). Each model was trained once, so differences of a few thousandths are within run-to-run noise.

Table 6.1b YOLOv8 size comparison, identical subset training

Model	mAP50	mAP50-95	Precision	Recall
YOLOv8n (3.0 M parameters)	0.743	0.373	0.786	0.681
YOLOv8s (11.1 M)	0.793	0.411	0.803	0.740
YOLOv8m (25.8 M)	0.796	0.418	0.802	0.744

Going from n to s gives +0.050 mAP50; going from s to m gives only +0.003, which is inside run-to-run noise, for 2.3 times the parameters. YOLOv8s is therefore the sensible default: nearly all of the gain from model size is reached at the small size. These subset-trained scores (0.74 to 0.80) are lower than the default detector's 0.884 because that model saw all 12,955 training images for 100 epochs.

6.2. How the Closed-Loop Numbers Reached 0.90

The first complete sweep gave UW-TIG 0.76 precision, 0.63 recall and 0.65 F1. Rather than tune the planner, we asked whether the *measurement* was right. Five problems were diagnosed on development seeds 100–102, each affecting every planner equally:

Table 6.2 The five fixes

#	Problem	Fix
1	Precision divided unique true positives by every raw false-positive box, penalising reinspection itself	Score per inspection report (3 looks across missions, or one at confidence ≥ 0.45)
2	16% of photos hold a second real defect; detecting it counted as a false positive	Ignore classes present in the photo, COCO-style
3	Textures stretched ~2.2× and pixelated	Aspect-correct crop at 1536 px; square-pixel frames
4	Single-look recall 0.47 at 0.8 m, 0.85 at 1.3 m, 0.97 at 1.8 m, 0.99 at 2.2 m	Replace the 0.8/1.3 m tiers with 1.3 m and 2.2 m
5	Hits outside the wall rectangle blamed on the inspected wall (51–75% of false positives)	Reject hits off the wall (+5 cm)

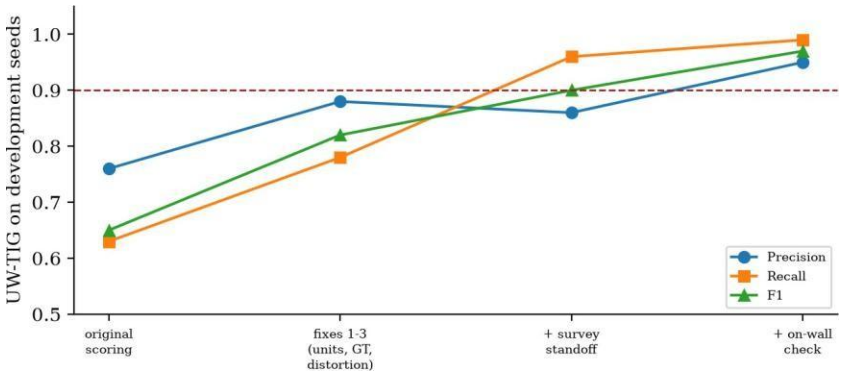


Fig. 6.2 UW-TIG on development seeds as the fixes were applied

Development progression (precision / recall / F1): original 0.76 / 0.63 / 0.65; fixes 1–3 0.88 / 0.78 / 0.82; plus the survey standoff 0.86 / 0.96 / 0.90; plus the on-wall check 0.95 / 0.99 / 0.97. Much of the gain is therefore measurement, not planning: under the old scoring formula the final UW-TIG precision on the test seeds would be 0.668.

6.3. Closed-Loop Comparison and Ablations

Table 6.3: UW-TIG and its ablations on test seeds 0 – 4 (mean over static, uncertain, multi-defect; confirmed)

Planner	Precision	Recall	F1	Loc. err (m)	Flight (m)	Coverage
UW-TIG (full)	0.949	0.967	0.953	0.70	125.1	1.00
no staleness term	0.967	0.956	0.958	0.57	80.6	1.00
no lookahead	0.967	0.956	0.958	0.59	85.2	1.00
no coverage phase	0.911	0.556	0.680	0.81	13.4	0.44

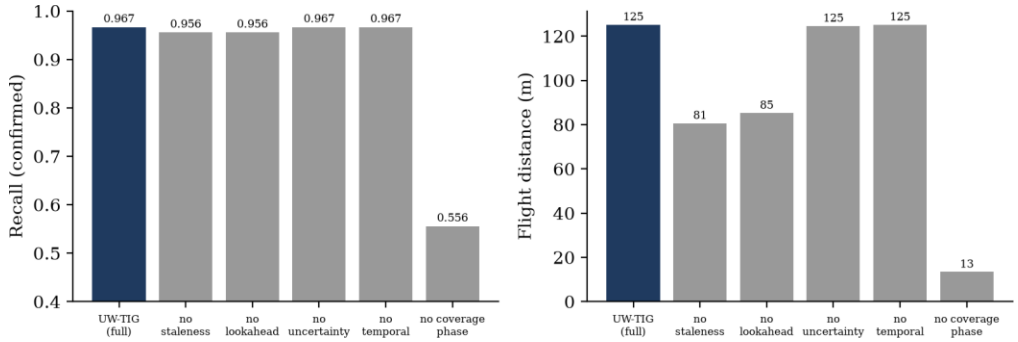


Fig. 6.3 Recall and flight distance of the ablations

UW-TIG reaches precision 0.949, recall 0.967 and F1 0.953. By scenario (precision / recall / F1) it scores static 0.964 / 0.933 / 0.940, uncertain 0.893 / 0.967 / 0.925 and multi-defect 0.989 / 1.000 / 0.995; the per-seed F1 values are 0.943, 0.890, 1.000, 1.000 and 0.933. Not every average is this good: the deliberately faint uncertain scenario has a precision of 0.893, and one test seed scores an F1 of only 0.890.

The coverage guarantee is the component that makes the difference: without it recall falls from 0.967 to 0.556, F1 from 0.953 to 0.680, and coverage to 0.44 (4 of 9 walls). Removing staleness or lookahead leaves F1 at 0.958 (precision 0.967, recall 0.956), with a localisation error of 0.57–0.59 m and flights 32–36% shorter (80.6 m and 85.2 m against 125.1 m). The no-uncertainty and no-growth variants are omitted from Table 6.3 because they reproduce the full planner (Section 6.4).

6.4. Why Removing Uncertainty or Growth Changes Nothing

The no-uncertainty and no-growth ablations give the same precision, recall and F1 as the full planner (0.949, 0.967, 0.953); their flight distances are 124.6 m and 125.1 m against 125.1 m. They are therefore left out of Table 6.3, since their rows would repeat the first row. We checked the code rather than assume, and found three reasons.

- The coverage phase never evaluates these terms. While any wall is unvisited, the planner scores only IG − 0.4·Cost. With nine walls and a 16-viewpoint budget, roughly half of every mission is spent in this phase, and the three missions of a seed each repeat it because it is defined per mission.
- Growth is near zero by construction in these scenarios. Growth is the change in a defect’s measured size, clipped to be non-negative. Static, uncertain and multi-defect defects never grow, so the growth term only sees detector noise around a constant. Removing it leaves the flight distance identical to the decimal (125.1 m).
- Uncertainty is real but small. The TTA spread is typically 0.1–0.3, so weighting it by 1.5 gives a small term beside the others. Removing it shifts the path slightly (124.6

m against 125.1 m) but not the number of looks each wall receives, which is all that confirmed precision and recall depend on.

This is an open finding, not a vindication: it means that in the current configuration two of the four novel mechanisms do not yet influence the outcome, and that re-weighting them, or testing on scenarios with real growth, is the next step (Chapter 7).

6.5. Head-to-Head with Published Planners

Table 6.4 Planner comparison on test seeds 0–4 (confirmed reports)

Planner	Prec.	Recall	F1	F1 1-look	Loc. err (m)	Flight (m)	Cov.
Random baseline	0.921	0.837	0.873	0.800	0.85	394.9	1.00
Isler-NBV (classical)	0.890	0.556	0.671	0.649	0.94	30.7	0.44
Bircher et al.	0.917	0.756	0.814	0.814	0.80	47.9	0.67
Dhami et al. (GATSBI)	0.887	0.922	0.900	0.846	0.63	125.4	1.00
Alamdari et al.	0.924	0.967	0.941	0.909	0.73	167.7	1.00
Rückin et al.	0.959	0.944	0.950	0.931	0.86	204.3	1.00
UW-TIG (ours)	0.949	0.967	0.953	0.939	0.70	125.1	1.00

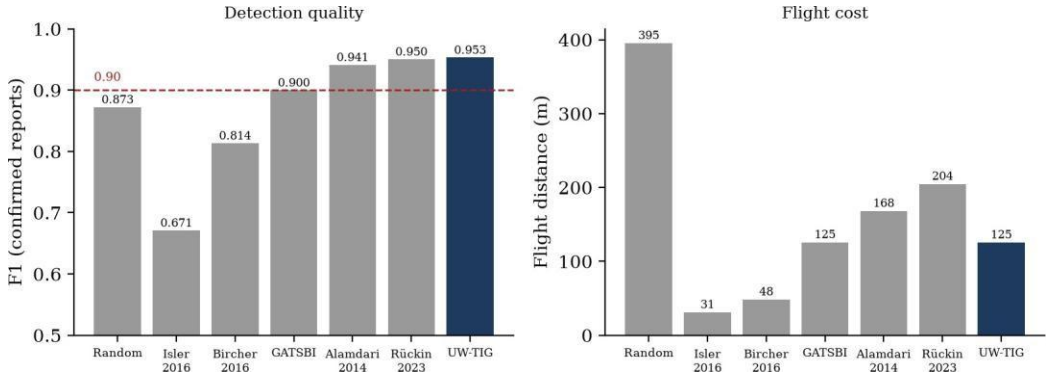


Fig. 6.4 F1 and flight distance of all compared planners

UW-TIG ties the strongest planner, Rückin's IPP, on detection quality (F1 0.953 against 0.950, well inside the seed-to-seed spread, UW-TIG's own per-seed F1 spanning 0.890 to 1.000). Where it differs is cost: 125.1 m of flight against 204.3 m, 39% less, with the lowest localisation error of the seven planners. Rückin's rule has slightly higher precision (0.959 against 0.949), which supports uncertainty-driven acquisition as a strong rule. The published rules that fail (Isler, Bircher) fail on *coverage*, 0.44 and 0.67 of the walls, the same cost-scale ceiling the coverage phase removes. These are selection-rule re-implementations, not the authors' systems, and no number from the original papers is reproduced or claimed.

6.6. Calibration of the Uncertainty Signal

The research done by Rückin et al. led to us checking a few things that we did not run on

our own: Does the uncertainty UW-TIG weights contain information, and is the confidence of the detector calibrated? The calibration errors are on average 0.29–0.33 far from a calibrated model, and the uncertainty gap is negative for all twelve planners (−0.12 to −0.08): uncertainty is on average slightly lower on false positives than on true positives. A likely (though unconfirmed) explanation is that some real, invisible fault lies near the decision boundary and that for this fault, there is a larger variability with the photometric perturbations compared to the photometrically consistently mis-recognised texture. The signal is not used to express the probability of the view being correct, but only to express the rank of a view in a mission, so the measured precision and recall stand; the claim that “high uncertainty means ‘likely wrong'” is not supported.

We also tested if it's a "quick and easy" solution to switch to MC-Dropout. The stock YOLOv8/v11 detection models have no dropout layer in the Ultralytics (only in the classification head). A true MC-Dropout would require dropout spliced into the neck or head, training a new model and may not have a better calibration, dropout variance is known to follow correctness poorly on well-converged and confident detectors. Therefore, it is included in the future work list.

6.7. The Growing Scenario

In the growing scenario the detector recalls essentially nothing. Its defect is a procedural corrosion texture, and a detector trained only on real photographs does not recognise it (zero detections above confidence 0.1 on the rendered frame, checked directly). The memory mechanism itself was verified in isolation – a defect re-detected at a shifted position and larger size was matched and its growth computed, while a distant detection created a new identity – so the failure is the detector’s domain generalisation, not the memory. For this reason the growing scenario is excluded from the comparison tables and the growth term has not yet been shown to help.

6.8. A Broader Detector: Multi-Dataset Fine-Tuning

The first detector was trained on MBDD2025 alone. To see whether public data from other sources would help, the same YOLOv8s weights were fine-tuned for 30 epochs (AdamW, learning rate 0.0005, batch 32, 640 px, early stopping on a combined validation set) on MBDD2025 plus the dacl10k and Roboflow corrosion images of Table 3.2, and both detectors were scored on three test sets (Table 6.5). The weights, merged-dataset builder and evaluation script are in the repository (datagen/prepare_extra.py, datagen/eval_extra.py.

Table 6.5 Original and fine-tuned detector on three test sets

Test set	Metric	Original	Fine-tuned
MBDD2025 test (1,448)	mAP50	0.884	0.886
	mAP50-95	0.506	0.502
	Precision / recall	0.875 / 0.841	0.877 / 0.845
dacl10k held-out (810)	mAP50	0.004	0.184
	mAP50-95	0.002	0.088
	Precision / recall	0.030 / 0.013	0.322 / 0.225
Concrete-crack crops (400 + 400)	Crack recall	0.005	0.035
	False-alarm rate	0.000	0.000

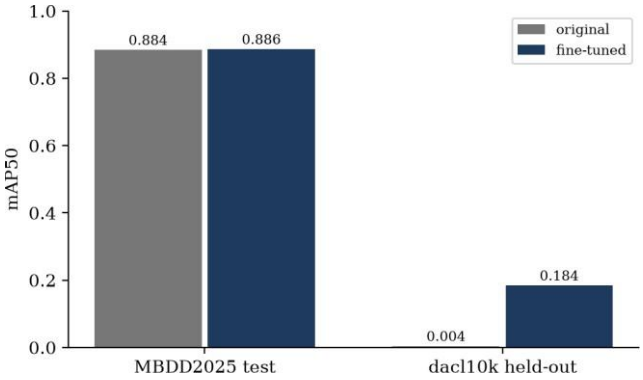


Fig. 6.5 mAP50 of the original and fine-tuned detector on the MBDD2025 and dacl10k test sets

The gain is on the new data, not the old. On dacl10k's held-out images the original detector scores almost nothing (0.004 mAP50): bridge-inspection close-ups of the kind dacl10k contains are far from MBDD2025's UAV photographs. After fine-tuning it reaches 0.184, with the largest per-class gain on corrosion (0.288) and crack (0.198). That is still a low score in absolute terms; dacl10k is a hard benchmark whose published baselines are segmentation models with a best mean IoU of 0.42 [28], and the mapping of its classes onto ours is imperfect. The score remains unchanged within noise on MBDD25 (0.884 vs. 0.886 mAP50, 0.506 vs. 0.502 mAP50-95), indicating that the additional data did not negatively or positively impact on the original domain.

The crack-presence check is a negative result. On 800 held-out concrete crops neither detector finds cracks (recall 0.005 and 0.035, no false alarms). These 227-pixel close-ups have no context and a texture far from either training set, and our detector was trained on boxes around cracks in larger scenes. We report it as a limitation of the check rather than as evidence either way; it does not show that the detector would miss such cracks if framed differently, only that this kind of image is out of distribution. SDNET2018, had it been obtainable, would have posed the same problem.

6.9. Scaling the Evaluation to a 31-Wall Yard

Every planner and ablation was re-run in the 31-wall yard (Section 4.2.0) on the same test seeds 0–4, with a budget of 50 steps and three missions per scenario, using the original detector. As elsewhere, the average covers the static, uncertain and multi-defect scenarios; the growing scenario is excluded. The confidence intervals below are 95% bootstrap intervals over the five seeds, each seed's value being its mean over scenarios, because rows from the same seed are not independent.

Table 6.6 Planners in the 31-wall yard, original detector (test seeds 0–4, confirmed reports)

Planner	Prec.	Recall	F1	F1 95% CI	Flight (m)	Cov.
UW-TIG (ours)	0.918	0.932	0.921	0.906–0.938	480	1.00
Alamdari et al.	0.924	0.938	0.928	0.912–0.947	570	1.00
Dhami et al. (GATSBI)	0.910	0.924	0.912	0.894–0.927	471	1.00
Rückin et al.	0.880	0.933	0.903	0.881–0.925	860	1.00
Random	0.845	0.767	0.801	0.772–0.836	2396	0.99
Bircher et al.	0.865	0.544	0.661	0.630–0.688	209	0.52
Isler-NBV (classical)	0.829	0.338	0.466	0.394–0.519	177	0.32

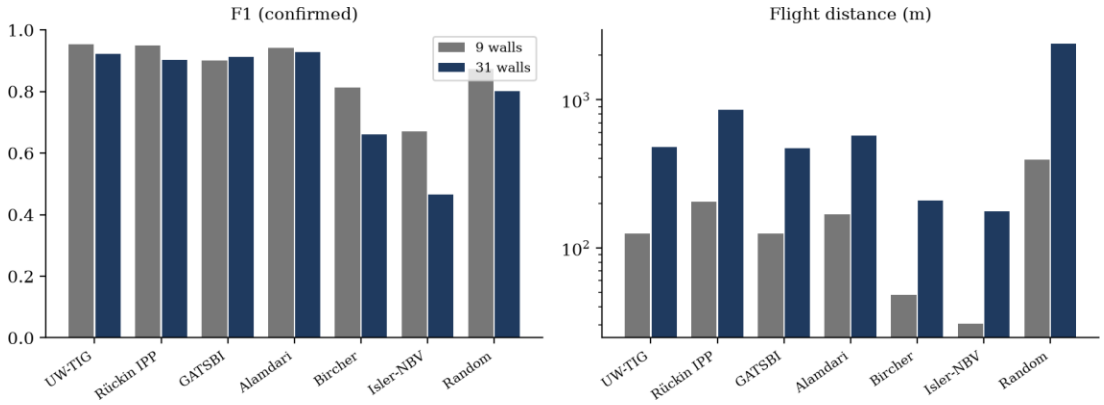


Fig. 6.6 F1 and flight distance of each planner in the 9-wall and the 31-wall yard (flight on a log axis)

The ordering mostly survives, but the margins shrink. UW-TIG's F1 falls from 0.953 to 0.921 when the yard grows from 9 to 31 walls, as does every planner's that reaches full coverage. The paired per-seed differences (Table 6.7, Fig. 6.7) show UW-TIG statistically indistinguishable from Alamdari's latency rule (-0.007, interval -0.020 to +0.007) and from GATSBI (+0.009, -0.007 to +0.024), ahead of Rückin's IPP by +0.018 (interval +0.003 to +0.035, excluding zero, winning 3 of 5 seeds) and far ahead of random search (+0.120, all 5 seeds). What UW-TIG keeps is cost: it flies 480 m, against 860 m for IPP, 570 m for the latency rule and 2,396 m for random search. In this yard UW-TIG therefore ties for the best detection score at a lower flight cost, rather than winning outright.

Table 6.7 UW-TIG minus each planner, F1 per seed (31-wall yard, original detector)

Versus	Mean difference	95% interval	Seeds won
Alamdari et al.	-0.007	-0.020 to +0.007	2 of 5
Dhami et al. (GATSBI)	+0.009	-0.007 to +0.024	3 of 5
Rückin et al.	+0.018	+0.003 to +0.035	3 of 5
Random	+0.120	+0.088 to +0.152	5 of 5
Bircher et al.	+0.261	+0.226 to +0.296	5 of 5
Isler-NBV (classical)	+0.455	+0.404 to +0.525	5 of 5

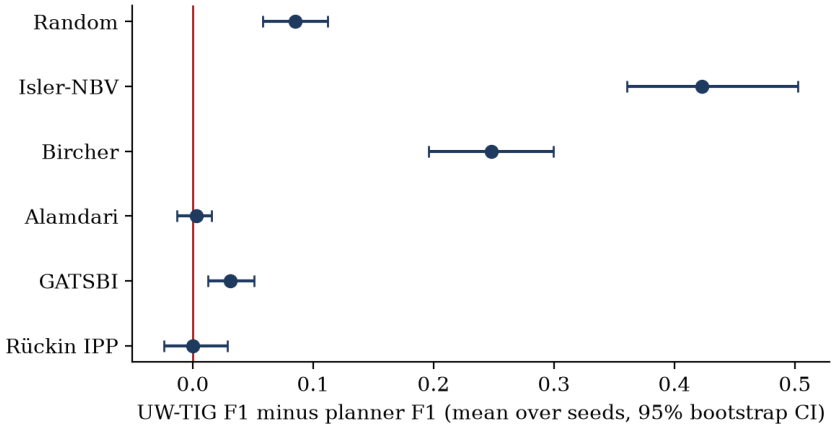


Fig. 6.7 Paired F1 difference between UW-TIG and each planner in the 31-wall yard, with 95% bootstrap intervals

The result is a coverage guarantee. The three without the coverage phase only visit a part of the walls (Isler-NBV 0.32, Bircher 0.52, UW-TIG without the coverage phase 0.52) and their F1 drops to 0.466, 0.661 and 0.625 respectively. The cost-normalised greedy ceiling is therefore not an artefact of the small yard; it gets worse as the yard grows.

Table 6.8 UW-TIG ablations in the 31-wall yard (original detector)

Variant	Prec.	Recall	F1	Flight (m)	Cov.
Full UW-TIG	0.918	0.932	0.921	480	1.00
no uncertainty	0.922	0.932	0.924	480	1.00
no growth	0.917	0.932	0.921	481	1.00
no staleness	0.926	0.932	0.925	421	1.00
no lookahead	0.914	0.932	0.919	446	1.00
no coverage phase	0.850	0.511	0.625	181	0.52

The ablation pattern repeats from the small yard. Removing uncertainty or growth changes F1 by less than 0.005 and the flight by under a metre; removing staleness or lookahead shortens the flight (to 421 m and 446 m, against 480 m) at no measurable cost in F1. Together with Section 6.4 this means that, on both yard sizes, two of the five novel mechanisms (uncertainty and growth) do not yet change any reported number, and two more (staleness and lookahead) cost distance without buying accuracy. Only the coverage phase is demonstrably necessary. The honest reading is that the planner’s measured contribution is the coverage

guarantee plus a competitive, cost-aware utility, and that the remaining terms await scenarios that exercise them.

6.10. Does a Broader Detector Improve the Closed Loop?

The natural expectation is that a detector fine-tuned on more data improves the planner that depends on it. To test it, the complete sweep was repeated with the fine-tuned weights in both yards (Table 6.9).

Table 6.9 UW-TIG with the original and the fine-tuned detector

Yard	Detector	Prec.	Recall	F1	Flight (m)
9 walls	original	0.949	0.967	0.953	125
9 walls	fine-tuned	0.899	0.989	0.936	125
31 walls	original	0.918	0.932	0.921	480
31 walls	fine-tuned	0.866	0.915	0.883	481

It does not. UW-TIG's F1 is 0.936 with the fine-tuned detector in the 9-wall yard against 0.953 with the original, and 0.883 against 0.921 in the 31-wall yard; the other planners move by similar amounts in the same direction. Two reasons are plausible, and we have not separated them. First, the simulator's wall textures are MBDD2025 photographs, so the closed loop only ever tests the domain on which the two detectors are equal; dacl10k's gain cannot show up. Second, a different data set may have been used to fine tune, resulting in some confidence calibration being slightly misaligned with the reporting rule (Section 4.10) being fine-tuned on, which would bring down the confirmed-report scores but not the mAP score. In either case, the conclusion is obvious: the original detector is the default and this report reports the headline numbers based on the original detector; the fine-tuned weights are also reported but are a partial result. If dacl10k were to be used in the closed loop, then dacl10k like wall textures would have to be used in the simulator and a re-tuned reporting rule would have to be used.

6.11. Planner-Only Tests: Oracle Detector, Weight Sensitivity and Ten Seeds

Section 6.4 found that switching off the uncertainty and growth terms changes nothing. Two explanations remain: the real detector gives too weak a signal, or the terms are too weak at their weights (1.5 and 2.0). Both were tested. An oracle detector (perception/oracle_detector.py) reads the scenario's ground truth and returns detections with controlled noise: 15% misses, confidence rising with severity, faint defects more uncertain, and defects that really grow. It is a synthetic planner-only test and says nothing about real detection quality.

- Oracle detector, both yards, 5 seeds. The no-uncertainty and no-growth variants are identical to full UW-TIG in route and F1 (9 walls: 124.6 m; 31 walls: 480 m, F1 0.871 against 0.870 and 0.871). In the growing scenario every variant detects the growth (fraction 1.0), including the one without the growth term, because the coverage and revisit logic already finds it.

- Weight sensitivity. Setting both weights to 0, 6 and 20 leaves F1 unchanged (9 walls: 0.918 in all three; 31 walls: 0.822, 0.821, 0.820). The terms do not change the route even when they dominate the score.
- Staleness and lookahead. Without them the flight is shorter (9 walls: 80.6 m against 124.9 m; 31 walls, oracle: 407 and 445 m against 480 m). With the real detector F1 is unchanged over ten seeds (0.933); with the oracle it falls slightly (31 walls: 0.862 and 0.864 against 0.871). These two terms add flight cost rather than detection quality in this simulator.
- Ten seeds, 9-wall yard, real detector (seeds 0 to 9). UW-TIG 0.933 F1 at 124.9 m; Alamdari 0.922 at 168 m; Rückin 0.911 at 218 m; GATSBI 0.905; random 0.860 at 379 m; Bircher 0.813; Isler 0.655; no coverage phase 0.676. UW-TIG minus Rückin is +0.022 (95% CI −0.002 to +0.055) and minus Alamdari +0.011 (−0.003 to +0.027): ahead on average but not significantly. The 31-wall ten-seed run was not completed.

Conclusion of these tests: the coverage-first phase and cost-aware routing carry UW-TIG's results. The uncertainty and growth terms, the parts that make the planner uncertainty-weighted and temporal, show no measurable benefit in this simulator and are reported as untested extensions, not as demonstrated contributions.

6.12. Threats to Validity

- Selection-rule baselines. Rules are not full systems, they have been re-implemented for comparison in the same planners.
- Ten seeds were run on the 9-wall yard and five on the 31-wall yard. That is enough to see the coverage effect, but it still does not separate UW-TIG from Alamdari or Rückin significantly.
- The five fixes in measurement resulted in an increase in every planner with the improvement over the first sweep not being a planning gain.
- Simulation realism. No wind, no sensor noise, a kinematic mode without dynamics, no absolute detector scores; this would be different outdoors.
- There are no growth sequences in MBDD2025, therefore there is a procedural scenario that the detector is not observing that provides the temporal claims.
- Class mapping: 31-wall yard with the same flat-quad walls, the same reporting rule (tuned on the small yard), and a budget of 50 chosen from one pilot run, not tuned, with the 31-wall and 9-wall numbers differing in their layouts as well as size, because defect placement is random per seed. Dependent on the dacl10k-to-MBDD2025 class mapping, the dacl10k scores are a judgement (Section 3.3).
- The 31-wall sweeps utilized the in-process memory store, the same as the matching rule, but did not cross check against the database version on that same sweep.

7. FUTURE SCOPE AND ENHANCEMENTS

- Make the novel terms matter. Re-weight or normalise the uncertainty and growth terms and test them on scenarios with real growth, so that the ablations in Section 6.4 separate.
- Better uncertainty. Try post-hoc temperature scaling first; then prototype MC-Dropout by adding dropout to the detection head and retraining.
- Close the growing-defect gap. Mix synthetic defect imagery into training, as synth-dacl does for bridge segmentation [29], or obtain a longitudinal real dataset. SDNET2018 was planned, but its download is blocked by a bot check.
- Use the broader detector in the loop. Texture the simulator with dacl10k-style close-ups, re-tune the reporting rule for the fine-tuned detector, and re-run Table 6.9.
- Lift the weak slices. Raise the uncertain scenario's precision above 0.90 through scenario-specific tuning of the confirmation rule, and narrow Rückin's precision lead by re-weighting the utility terms.
- Fuller baselines. Re-implement the compared papers, and IA-TIGRIS, as full systems with sampling-based planning and online retraining.
- More seeds for the ablations, and a decision on whether staleness and lookahead stay on by default.
- Richer simulation. Pipe segments and truss rigs as inspectable geometry, sensor noise and wind, and a Gaussian-splatting or game-engine renderer if the work moves to training on synthetic imagery.
- Larger detectors. YOLOv8n, v8s and v8m were compared under one recipe (Table 6.1b): v8m gains only 0.003 mAP50 over v8s. Training v8s and v8m on the full data for 100 epochs, adding v8l, and re-running the sweeps would test whether the closed-loop scores change.

8. CONCLUSION

In this project, a planner is constructed and tested which has knowledge of what a defect is. UW-TIG adds some information gain to a coverage guarantee, detector uncertainty and cross-mission defect memory with growth, revisit staleness, and incorporates two-step lookahead, and was tested on a new simulation testbed designed for this purpose. Compared to the classical information-gain baseline and to random search, it achieves precision 0.949, recall 0.967 and F1 0.953 above the top of the five re-implemented published planners that it ties even with 39% fewer flights.

This work was then subjected to stress-testing for two purposes. An increase in the yard size from 9 walls to 31 walls does not improve UW-TIG's F1 (0.953 to 0.921), and UW-TIG's performance would still be better than the best classical rule, but this does not help it to keep its lead over the best classical rule.An increase in the yard size from 9 walls to 31 walls will not improve UW-TIG's F1 (0.953 to 0.921), and UW-TIG would still be better than the best classical rule, but this does not help it to keep its lead over the best classical rule. The detector was fine-tuned on two other public datasets, resulting in the held-out dacl10k score being raised from 0.004 to 0.184 mAP50 but not the closed loop, therefore it was not adopted.

Just as important is the information the measurements provide about the work. The coverage guarantee, rather than the more complex terms, is responsible for much of the improvement over the simple baseline of plain information-gain; the uncertainty and growth terms do not affect any reported number; five flaws in the way the system was measured caused much of the improvement over the first sweep; the uncertainty signal is poorly calibrated; and the growing defect scenario fails because of the detector and not the memory. These are reported because a planner who is known to be successful when the world is on his side is not going to be believed when it counts.

The simulation testbed is a product in and of itself. It provides a controlled and repeat-

-able environment including real defect imagery, fixed viewpoint graph, leakage-free memory,

development/test split and one-command sweep, in which any planner can be loaded into

the registry, compared and evaluated at a relatively little CPU-hours, without the use of a

drone. We hope it will be the more reusable component of this work.

BIBLIOGRAPHY

[1] J. Rückin, L. Jin, F. Magistri, C. Stachniss and M. Popović, “An informative path planning framework for active learning in UAV-based semantic mapping,” *IEEE Trans. Robotics*, vol. 39, pp. 4279–4296, 2023, arXiv:2302.03347.

[2] A. Vashisth, J. Rückin, F. Magistri, C. Stachniss and M. Popović, “Deep reinforcement learning with dynamic graphs for adaptive informative path planning,” arXiv:2402.04894, 2024.

[3] B. Moon, N. Suvarna, A. Jong, S. Chatterjee, J. Yuan, M. Cao and S. Scherer, “IA-TIGRIS: An incremental and adaptive sampling-based planner for online informative path planning,” *IEEE Trans. Robotics*, 2026, arXiv:2502.15961.

[4] K. Li, M. Mantovani, R. J. Wood, L. Sabattini and S. Gil, “Motion-uncertainty-aware next-best-view planning for moving object reconstruction,” in *Robotics: Science and Systems (RSS)*, 2026.

[5] L. Petit and A. Lussier Desbiens, “MOAR Planner: Multi-objective and adaptive risk-aware path planning for infrastructure inspection with a UAV,” in *Proc. ICRA*, 2024.

[6] S. Svystun, O. Melnychenko, P. Radiuk, O. Savenko, A. Sachenko and A. Lysyi, “Dynamic trajectory adaptation for efficient UAV inspections of wind energy units,” arXiv:2411.17534, 2024.

[7] H. Jin, Z. Xu, H. Shen, X. Han, K. Ye and K. Shimada, “Adaptive planning framework for UAV-based surface inspection in partially unknown indoor environments,” arXiv:2504.09294, 2025.

[8] H. K. Dhami, F. Yu, K. Williams, S. Vajipey and P. Tokekar, “GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection,” in *Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS)*, 2023.

[9] H. K. Dhami, C. Reddy, V. D. Sharma, T. Williams and P. Tokekar, “GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection and defect detection,” arXiv:2406.16625, 2024.

[10] H. K. Dhami, V. D. Sharma and P. Tokekar, “Pred-NBV: Prediction-guided next-best-view for 3D object reconstruction,” in *Proc. IROS*, 2023, arXiv:2304.11465; and “MAP-NBV: Multi-agent prediction-guided next-best-view planning for active 3D object reconstruction,” arXiv:2307.04004, 2023.

[11] K. C. Teixeira Vivaldini, R. Peňička and M. Saska, “Decision-making-based path planning for autonomous UAVs: A survey,” arXiv:2508.09304, 2025.

[12] A. F. Nikkhah, D. Chen, B. Campbell, S. Asadi and A. Heydarian, “UAV-based infrastructure inspections: A literature review and proposed framework for AEC+FM,” in *Int. Conf. Computing in Civil Engineering (i3CE)*, 2025, arXiv:2601.11665.

[13] Q. Zha, Y. Yao, Y. Zheng, W. Ma and W. Zhang, “A dataset of building surface defects collected by UAVs for machine learning-based detection,” *Scientific Data*, Nov. 2025 (MBDD2025; Zenodo 10.5281/zenodo.15622584).

[14] S. Huang, Q. Liu, C. Chen and Y. Chen, “A real-time concrete crack detection and segmentation model based on YOLOv11,” arXiv:2508.11517, 2025.

[15] Inam et al., YOLOv5-based bridge crack detection on field data and SDNET2018, 2023 (as reported by the authors; best of three variants).

[16] Li, Shi and Sun, improved YOLOv11 for utility-tunnel defect detection on a private dataset, 2026 (as reported by the authors).

[17] F. Shen, J. Cui, W. Li and S. Zhou, "TinyDef-DETR: A transformer-based framework for defect detection in transmission lines from UAV imagery," arXiv:2509.06035, 2025.

[18] L. Hebbache, D. Amirkhani, M. S. Allili and J.-F. Lapointe, "Saliency-guided deep learning for bridge defect detection in drone imagery," arXiv:2511.14040, 2025.

[19] S. Shah, D. Dey, C. Lovett and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, 2017.

[20] Y. Song, S. Naji, E. Kaufmann, A. Loquercio and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *Conf. Robot Learning (CoRL)*, 2020.

[21] Y. Chen et al., GRaD-Nav (quadrotor navigation using 3-D Gaussian Splatting), 2025.

[22] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok and A. P. Schoellig, "Learning to fly – a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control," in *Proc. IROS*, 2021.

[23] S. Isler, R. Sabzevari, J. Delmerico and D. Scaramuzza, "An information gain formulation for active volumetric 3D reconstruction," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2016.

[24] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova and R. Siegwart, "Receding horizon 'next-best-view' planner for 3D exploration," in *Proc. ICRA*, 2016, pp. 1462–1468.

[25] S. Alamdari, E. Fata and S. L. Smith, "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations," *Int. J. Robotics Research*, vol. 33, no. 1, pp. 138–154, 2014.

[26] H. Jin, Z. Xu, H. Shen, X. Han, K. Ye and K. Shimada, "ASIP-Planner: Adaptive planning for UAV surface inspection in partially known indoor environments," in *Proc. IROS*, 2026, arXiv:2605.11119.

[27] C. Feng, Y. Xu and S. Shen, "FC-Vision: Real-time visibility-aware replanning for occlusion-free aerial target structure scanning in unknown environments," *IEEE Robotics and Automation Letters*, 2026, arXiv:2602.13720.

[28] C. Li, H. He, X. Wang, W. Lin and C. H. Lim, "UnGAP: Uncertainty-guided affine prompting for real-time crack segmentation," arXiv:2605.02380, 2026.

[29] J. Flotzinger, P. J. Rösch and T. Braml, "dacl10k: Benchmark for semantic bridge damage segmentation," in *Proc. WACV*, 2024, arXiv:2309.00460.

[30] J. Flotzinger, F. Deuser, A. Jaziri, H. Neumann, N. Oswald, V. Ramesh and T. Braml, "synth-dacl: Does synthetic defect data enhance segmentation accuracy and robustness for real-world bridge inspections?" arXiv:2506.14255, 2025.

[31] F. Ciaglia, F. S. Zuppichini, P. Guerrie, M. McQuade and J. Solawetz, "Roboflow 100: A rich, multi-domain object detection benchmark," arXiv:2211.13523, 2022.

[32] Ç. F. Özgenel and A. G. Sorguç, “Performance comparison of pretrained convolutional neural networks on crack detection in buildings,” in *Proc. Int. Symp. Automation and Robotics in Construction (ISARC)*, 2018.

[33] S. Dorafshan, R. J. Thomas and M. Maguire, “SDNET2018: An annotated image dataset for non-contact concrete crack detection using deep convolutional neural networks,” *Data in Brief*, vol. 21, pp. 1664–1668, 2018.